

Specialized Experiment 1

Short-term Rainfall Forecasting & Alert System

Week 5 Session A | Duration: 2 hours | Smart Water Lab Series

Experiment Overview

This experiment focuses on building a real-time rainfall monitoring system that integrates external weather APIs, implements threshold-based alerting logic, and displays results through a web dashboard. You will apply AI-assisted development techniques to create a functional system for urban flood management.

Learning Objectives

- Integrate external weather APIs using AI assistance
- Implement threshold-based alerting logic
- Build a real-time monitoring dashboard with Streamlit
- Apply domain knowledge to validate results
- Document AI interactions in the Prompt Log

Physical Background

Short-term rainfall forecasting (Nowcasting) typically covers 0-6 hours ahead and is critical for urban flood management. The China Meteorological Administration defines rainfall intensity categories as follows:

Category	Intensity (mm/h)	Color Code
Light	< 2.5	Green
Moderate	2.5 - 8	Blue
Heavy	8 - 16	Yellow
Violent	≥ 16	Red

For this experiment, we use a threshold of 20 mm/h as the alert trigger, which corresponds to heavy rainfall.

Prerequisites

- Completed Weeks 1-4 of the course
- Understanding of Python basics and API concepts
- OpenCode Cloud environment set up

- Free OpenWeatherMap API key (obtain at openweathermap.org)

Materials and Tools

- OpenCode Cloud IDE
- Python 3.10+
- Required libraries: requests, pandas, streamlit
- OpenWeatherMap API key
- Sample data for testing

Experiment Structure

Part 1: API Integration (30 min)

Task: Use AI to generate code for fetching weather data from OpenWeather API.

Sample Prompt: I am a water resources student building a rainfall monitoring system. Please write Python code to fetch current weather data for Beijing using the OpenWeatherMap API. The code should: 1. Use the requests library to make the API call 2. Extract rainfall intensity from the response 3. Handle API errors gracefully 4. Include comments explaining each step
API endpoint: <https://api.openweathermap.org/data/2.5/weather>

Part 2: Alert Logic Implementation (30 min)

Task: Implement threshold-based alerting based on rainfall intensity.

Requirements: - Green: Rainfall < 10 mm/h (Normal) - Yellow: $10 \leq \text{Rainfall} < 20$ mm/h (Moderate) - Red: Rainfall ≥ 20 mm/h (Heavy - ALERT) When Red alert triggers: - Display warning message - Log event to file with timestamp - Show alert in dashboard

Part 3: Dashboard Creation (40 min)

Task: Build a Streamlit dashboard to display rainfall data and alerts.

Dashboard Requirements: 1. Title: 'Rainfall Monitor - [City Name]' 2. Current rainfall display (large metric) 3. Alert status indicator (color-coded) 4. Historical data chart (if time permits) 5. Auto-refresh every 5 minutes

Part 4: Testing & Validation (20 min)

Task: Verify the system works correctly and validate physical reasonableness.

1. Test with different locations (compare rainfall values)
2. Verify alert triggers at correct threshold
3. Check log file contains proper timestamps
4. Validate: Are rainfall values physically reasonable for the location?
5. Document any AI errors found and corrections made

Deliverables

- `weather_monitor.py` - Main application code
- `alert_log.txt` - Log of all triggered alerts
- `prompt_log.md` - Documentation of AI interactions
- Screenshot of working dashboard

Grading Criteria

Criterion	Points	Weight
API Integration (code works, handles errors)	25	25%
Alert Logic (correct thresholds, proper logging)	25	25%
Dashboard (functional, user-friendly)	25	25%
Prompt Log (documented AI interactions)	15	15%
Code Quality (comments, structure)	10	10%

Sample Solution Structure

```
# weather_monitor.py - Structure
import requests
import streamlit as st
from datetime import datetime

API_KEY = 'your_api_key'
CITY = 'Beijing'

def fetch_weather(city):
    # AI-generated: API call implementation
    pass

def check_alert(rainfall):
    # Your implementation: threshold logic
    pass

def log_alert(rainfall, level):
    # Your implementation: logging
    pass

# Streamlit dashboard
st.title('Rainfall Monitor')
# ... dashboard code
```

Tips for Success

- Start by testing the API call in isolation
- Use print statements to debug before building the dashboard
- The free OpenWeather tier has rate limits (60 calls/min)
- If API returns 0 rainfall, it might not be raining (expected behavior)
- Always verify AI-generated code handles errors gracefully

Optional Extensions

- Add multiple city monitoring
- Implement email/SMS notifications for alerts
- Create rainfall prediction using historical trends
- Add map visualization with Folium